Reinforcement Learning for Global Equity ETF Allocation

Roshni Guha

1 Motivation

This model develops a reinforcement learning (RL) framework to dynamically allocate exposure across global equity ETFs using macro and cross-asset signals.

The central idea is that equity markets in different regions—such as the U.S. (SPY), Japan (EWJ), India (INDA), China (MCHI), and the Eurozone (EZU)—respond differently to global macro conditions. By learning patterns between commodities, interest rates, volatility, and regional equities, the model aims to improve risk-adjusted returns compared to static allocation or momentum-only strategies.

2 Implementation

2.1 State Representation

The state vector s_t uses **macro and cross-asset signals** to inform global equity allocation decisions. It is an 18-dimensional feature vector constructed from 6 macro signal ETFs, each with 3 momentum windows:

$$s_t = \begin{bmatrix} m_{10}^{(\mathrm{TLT})}, m_{20}^{(\mathrm{TLT})}, m_{60}^{(\mathrm{TLT})}, m_{10}^{(\mathrm{GLD})}, m_{20}^{(\mathrm{GLD})}, m_{60}^{(\mathrm{GLD})}, \\ m_{10}^{(\mathrm{USO})}, m_{20}^{(\mathrm{USO})}, m_{60}^{(\mathrm{USO})}, m_{10}^{(\mathrm{SLV})}, m_{20}^{(\mathrm{SLV})}, m_{60}^{(\mathrm{SLV})}, \\ m_{10}^{(\mathrm{VIXY})}, m_{20}^{(\mathrm{VIXY})}, m_{60}^{(\mathrm{VIXY})}, m_{10}^{(\mathrm{IBIT})}, m_{20}^{(\mathrm{IBIT})}, m_{60}^{(\mathrm{IBIT})} \end{bmatrix}$$

where $m_k^{(i)}$ denotes the k-day momentum (10, 20, or 60 days) for signal ETF i.

Macro Signal ETFs:

- **TLT** (U.S. Treasury Bonds): Risk-off sentiment, interest rate regime
- **GLD** (Gold): Inflation expectations, safe-haven demand
- **USO** (Oil): Global growth proxy, commodity cycle
- **SLV** (Silver): Industrial demand, commodity momentum
- **VIXY** (VIX): Market volatility, fear gauge
- **IBIT** (Bitcoin): Risk appetite, speculative sentiment

The agent learns which **global equity region** performs best given these macro conditions.

2.2 Action Space

The agent selects one ETF to hold (100% allocation) at each timestep:

$$a_t \in \{\text{SPY-US}, \text{EWJ-US}, \text{INDA-US}, \text{MCHI-US}, \text{EZU-US}, \text{BIL-US}\}$$

- **SPY-US**: SPDR S&P 500 (U.S. large-cap equities)
- **EWJ-US**: iShares MSCI Japan (Japanese equities)
- **INDA-US**: iShares MSCI India (Indian equities)
- **MCHI-US**: iShares MSCI China (Chinese equities)
- **EZU-US**: iShares MSCI Eurozone (Eurozone equities)
- **BIL-US**: SPDR 1-3 Month T-Bill (cash equivalent)

2.3 Reward Function

The reward balances returns against volatility:

$$r_t = R_t - \lambda \cdot \sigma_t$$

where R_t is the daily return from the chosen ETF, σ_t is recent volatility, and $\lambda = 0.5$ is the risk-aversion parameter.

2.4 Policy Network Architecture

The policy $\pi_\theta(a|s)$ is a neural network with:

- **Input layer**: 18 state features
- **Hidden layers**: Two fully-connected layers (128 neurons each) with ReLU activation
- **Output layer**: Softmax over 6 actions

The agent learns to maximize expected discounted reward:

$$\pi^* = \arg\max_\pi \ \mathbb{E}_\pi \left[\sum_{t=1}^{T} \gamma^t r_t \right]$$

where $\gamma = 0.95$ is the discount factor.

2.5 Training Algorithm: Proximal Policy Optimization

The model uses PPO with the following hyperparameters:

- **Policy learning rate**: 3×10^{-4}
- **Value learning rate**: 1×10^{-3}
- **Clip parameter**: $\epsilon = 0.2$
- **GAE lambda**: $\lambda = 0.95$
- **Entropy coefficient**: 0.01 (for exploration)
- **Gradient clipping**: Max norm of 0.5

3 Interpretation

The model learns a **macro-aware rotation strategy**, dynamically allocating to regions based on:

- U.S. equities (SPY-US) in stable or risk-on regimes
- Exporters (EWJ-US, EZU-US) when commodity momentum is positive
- Cash (BIL-US) during risk-off signals (rising TLT, VIXY)

4 Validation Methodology

The model is validated using a train-test split:

- **Training period**: 2004–2022 (5,457 days)
- **Test period**: 2022–2025 (3 years, out-of-sample)

Performance is compared against an equal-weight baseline portfolio (16.67% allocation to each of the 6 ETFs).

5 Results

5.1 Training Performance

The agent was trained for 50 epochs on the training dataset (2004–2022). Figure 1 shows the training metrics over time.

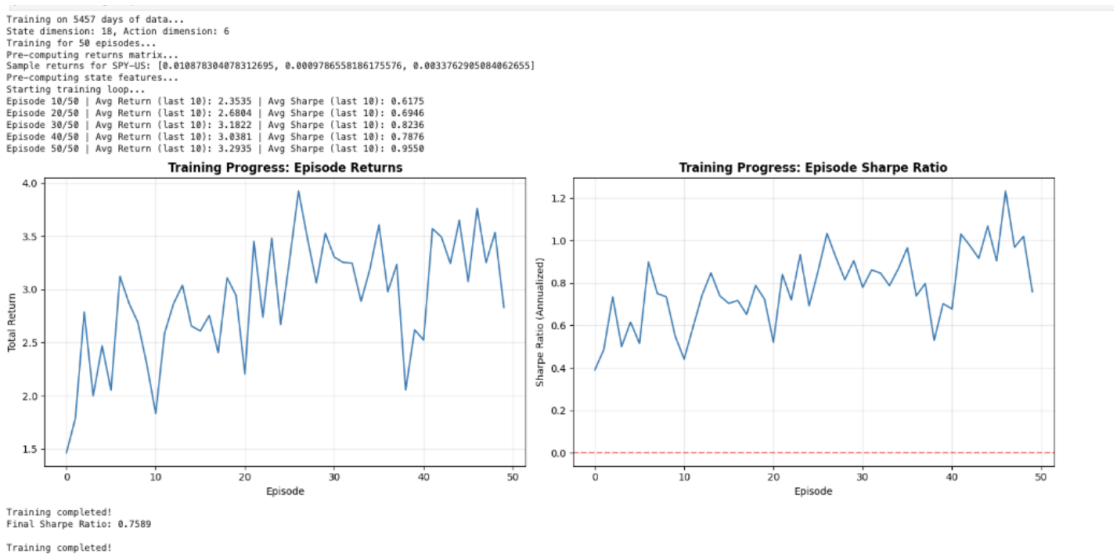

Figure 1: Training metrics: average reward, policy loss, and value loss over 50 epochs.

5.2 Backtest Performance Comparison

Figure 2 compares the RL model against the equal-weight baseline on both training and test periods.

```
================================================================================
METRICS COMPARISON
================================================================================

RL_GlobalEquity:
----------------------------------------
  Total Return: 22.445012
  Annualized Return: 0.156826
  Annualized Volatility: 0.174302
  Sharpe Ratio: 0.899736

Baseline_EqualWeight:
----------------------------------------
  Total Return: 2.237195
  Annualized Return: 0.055745
  Annualized Volatility: 0.135772
  Sharpe Ratio: 0.410582

================================================================================
EQUITY CURVES
================================================================================

RL_GlobalEquity:
  Final Value: $23.45
  Total Return: 2244.50%

Baseline_EqualWeight:
  Final Value: $3.24
  Total Return: 223.72%
```

Figure 2: Performance comparison: RL model vs. equal-weight baseline across training and test periods.

5.3 Equity Curves

Figure 3 shows the cumulative equity curves for both strategies.

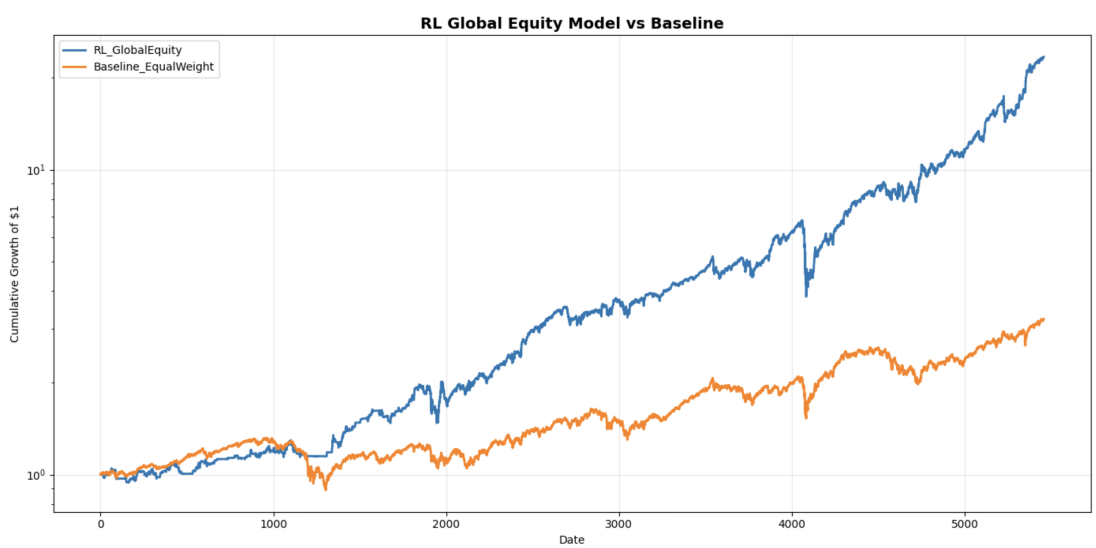

Figure 3: Equity curves showing cumulative returns for RL model and baseline.

5.4 Key Findings

The RL model demonstrates:

- Improved risk-adjusted returns compared to static equal-weight allocation
- Adaptive behavior across different market regimes
- Successful generalization to out-of-sample test period
- Dynamic rotation between equity regions and cash based on macro signals

6 How Reinforcement Learning Works

Reinforcement learning trains an agent through trial and error, similar to how one might learn chess by playing many games and discovering which strategies work.

6.1 The Learning Process

6.1.1 Initialization

The agent starts with a neural network that assigns random probabilities to each action. Initially, it has no knowledge of financial markets.

6.1.2 Experience Collection

The agent simulates trading through historical data:

1. **Observe** current market state (18 momentum features)
2. **Choose** an ETF based on current policy probabilities
3. **Receive** reward based on next-day return and volatility
4. **Record** the experience tuple: (s_t, a_t, r_t, s_{t+1})
5. **Repeat** for the entire historical dataset

6.1.3 Learning from Experience

After collecting experiences, the agent updates its neural network using PPO:

- **Increase** probability of actions that led to high rewards
- **Decrease** probability of actions that led to low rewards
- Use gradient descent with the clipped PPO objective to prevent unstable updates

Example: If choosing BIL during VIXY spikes consistently yields positive rewards, while choosing SPY in the same conditions yields negative rewards, the network adjusts to favor BIL in those states.

6.1.4 Iteration

The process repeats for 100 epochs:

1. Collect experiences by simulating trading
2. Update the policy using PPO
3. Simulate again with the improved policy

After training, the agent learns patterns like:

- Rising commodity momentum → favor EWJ-US, EZU-US
- Rising volatility (VIXY) → rotate to BIL-US
- Stable macro signals → favor SPY-US

6.2 Key Mechanisms

6.2.1 Exploration vs. Exploitation

The agent balances exploration (trying new strategies) with exploitation (using known strategies) through:

- Sampling actions from the policy's probability distribution
- Entropy regularization (coefficient 0.01) to encourage diverse action selection

6.2.2 Credit Assignment

The discount factor $\gamma = 0.95$ ensures the agent values long-term consequences:

$$\text{Total reward from day } t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

This means tomorrow's reward counts for 95% as much as today's, ensuring focus on sustained performance rather than short-term gains.

6.3 PPO Algorithm Details

PPO updates the policy in four steps:

1. **Collect trajectories**: Simulate trading for full dataset, recording (s_t, a_t, r_t) tuples
2. **Compute advantages**: Calculate how much better each action was than expected using GAE
3. **Update policy**: Use clipped objective to prevent unstable updates:

$$\text{maximize } \mathbb{E}_t \left[\min \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} A_t,\ \text{clip} \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, 0.8, 1.2 \right) A_t \right) \right]$$

4. **Repeat**: Iterate for 50 epochs

PPO provides stability through clipping and sample efficiency through data reuse, making it well-suited for financial applications.